Processamento de Alto Desempenho
Atividade Remota

Otimização Sequencial, Hierarquia de Memória e Cache

Nível: médio a alto | **Modalidade:** individual | **Linguagem:** Python

---

1. Objetivos

Esta atividade tem como objetivo investigar experimentalmente como decisões aparentemente simples de programação podem afetar significativamente o desempenho de um programa.

Ao final da atividade, o aluno deverá ser capaz de:

- identificar cálculos redundantes e operações desnecessárias;
- comparar diferentes implementações de um mesmo algoritmo;
- medir corretamente o tempo de execução;
- relacionar complexidade algorítmica e desempenho observado;
- explicar o papel da hierarquia de memória;
- compreender os conceitos de cache L1, L2 e L3;
- identificar localidade espacial e temporal;
- compreender o efeito de *cache hit* e *cache miss*;
- analisar como a ordem de acesso aos elementos de uma matriz afeta o desempenho;
- distinguir um problema predominantemente *compute-bound* de um problema *memory-bound*.

2. Parte I – Análise conceitual

Questão 1 – Otimização não significa apenas "fazer menos contas"

Considere dois programas que realizam exatamente o mesmo cálculo. O programa A executa menos operações aritméticas que o programa B. Entretanto, experimentalmente, o programa B termina mais rapidamente.

Explique como isso pode acontecer. Sua resposta deve obrigatoriamente discutir pelo menos quatro dos seguintes aspectos:

- hierarquia de memória;
- cache;
- localidade espacial;
- localidade temporal;
- acesso sequencial à memória;
- acesso aleatório à memória;
- chamadas de funções;
- estruturas de dados;
- vetorização;
- interpretação Python versus código compilado.

Extensão: dê um exemplo concreto de situação em que reduzir operações matemáticas pode não resultar em redução proporcional do tempo de execução.

Questão 2 – Hierarquia de memória

Considere, de forma simplificada, a seguinte hierarquia:

Registradores → L1 → L2 → L3 → RAM

Responda:

a) Por que o processador utiliza diferentes níveis de memória em vez de acessar diretamente a RAM?
b) O que significa dizer que ocorreu um *cache hit*?
c) O que significa um *cache miss*?
d) Explique a diferença entre localidade espacial e localidade temporal.
e) Compare os padrões de acesso abaixo:

```
A[0], A[1], A[2], A[3], A[4], ...
```

```
A[0], A[1000], A[2000], A[3000], ...
```

Qual deles tende a utilizar melhor a cache? Explique.

3. Parte II – Otimização sequencial

Considere a seguinte função:

```python
import math

def calcula(x):
    resultado = []

    for i in range(len(x)):
        valor = math.sqrt(x[i]) + math.sqrt(x[i])
        resultado.append(valor)

    return resultado
```

Questão 3 – Análise do código

Identifique pelo menos **três possíveis fontes de ineficiência** nesse código. Para cada uma:

1. identifique o problema;
2. explique por que ele pode afetar o desempenho;
3. proponha uma modificação.

Questão 4 – Três implementações

Implemente três versões diferentes da função anterior:

- **Versão A – referência:** mantenha a implementação original.
- **Versão B – otimizada em Python:** elimine cálculos redundantes e faça outras otimizações que considerar adequadas.
- **Versão C – NumPy:** implemente uma versão vetorizada utilizando NumPy.

Teste os três programas utilizando vetores com aproximadamente

$$N \in \left\{10^4, 10^5, 10^6, 10^7\right\}.$$

Caso sua máquina não suporte algum tamanho, registre essa limitação no relatório.

Utilize:

```python
from time import perf_counter
```

Para cada tamanho, realize pelo menos **cinco execuções**. Não utilize apenas uma medição. Calcule menor tempo, maior tempo, tempo médio e mediana.

N	Versão A	Versão B	Versão C
10^4			
10^5			
10^6			
10^7			

Questão 5 – Speedup

Considere a versão A como referência. Calcule o *speedup* por

$$\boxed{S = \frac{T_A}{T_{\text{otimizado}}}}$$

para as versões B e C.

Responda:

a) Qual versão apresentou maior *speedup*?
b) O *speedup* permaneceu aproximadamente constante quando N aumentou?
c) Caso não tenha permanecido, proponha uma explicação.
d) A versão NumPy deve ser considerada apenas uma "otimização do código Python"? Explique.

4. Parte III – Cache e organização da memória

Considere agora uma matriz quadrada:

```
import numpy as np

N = 3000
A = np.random.random((N, N))
```

Algoritmo 1 – percurso por linhas

```
soma = 0.0
for i in range(N):
    for j in range(N):
        soma += A[i, j]
```

Algoritmo 2 – percurso por colunas

```
soma = 0.0
for j in range(N):
    for i in range(N):
        soma += A[i, j]
```

Questão 6 – Previsão antes do experimento

Antes de executar o código, responda: qual dos dois algoritmos você acredita que será mais rápido?

Justifique utilizando os conceitos de organização da matriz na memória, C-order, linhas de cache, localidade espacial e *cache miss*. Essa resposta deverá ser escrita **antes de observar os tempos obtidos**.

Questão 7 – Experimento

Execute os dois algoritmos para diferentes valores, por exemplo:

$$N \in \{500, 1000, 2000, 3000, 4000\}.$$

Ajuste os maiores tamanhos caso sua máquina apresente limitações. Realize pelo menos três execuções de cada caso e apresente os resultados em tabela.

Calcule também

$$\boxed{R = \frac{T_{\text{colunas}}}{T_{\text{linhas}}}}$$

e interprete o significado de R.

5. Parte IV – Investigação do stride

Considere:

```
import numpy as np
from time import perf_counter

N = 20_000_000
A = np.ones(N)

for stride in [1, 2, 4, 8, 16, 32, 64, 128, 256, 512]:
    soma = 0.0
    inicio = perf_counter()

    for i in range(0, N, stride):
        soma += A[i]

    fim = perf_counter()
    print(stride, fim - inicio)
```

Questão 8 – Um resultado que pode enganar

Executar esse código e simplesmente comparar os tempos pode levar a uma conclusão incorreta. Explique por quê.

Dica: quando o *stride* aumenta, o número total de elementos processados também muda.

Questão 9 – Métrica adequada

Modifique o experimento de maneira que seja possível calcular

$$\boxed{t_{\text{elemento}} = \frac{T}{N_{\text{acessos}}}}$$

ou outra métrica equivalente.

Construa um gráfico de **tempo médio por acesso versus stride** e analise o resultado. Sua discussão deverá considerar linha de cache, quantidade de dados aproveitados em cada carregamento, localidade espacial, desperdício de dados carregados e possíveis limitações do experimento realizado em Python.

6. Parte V – Problema avançado

Questão 10 – Matrizes muito grandes

Considere uma aplicação científica que trabalha repetidamente com uma matriz de

$$20\,000 \times 20\,000$$

elementos `float64`, sendo cada elemento armazenado em 8 bytes.

a) Estime a quantidade de memória necessária apenas para armazenar essa matriz. Apresente o cálculo.
b) É razoável esperar que essa matriz inteira permaneça na cache L3 de um processador convencional?
c) Qual será provavelmente o principal gargalo durante percursos repetidos sobre essa matriz?
d) Esse problema tende a ser *compute-bound*, *memory-bound*, ou depende do algoritmo? Justifique.

Como referência, a memória ocupada pode ser estimada por

$$M = N_{\text{linhas}}\, N_{\text{colunas}}\, b,$$

onde b é o número de bytes por elemento.

7. Parte VI – Desafio de otimização

Questão 11 – Mudança da ordem dos laços

Considere a multiplicação de matrizes

$$\boxed{C = A\,B}.$$

Uma implementação elementar utiliza três laços:

```
for i in range(N):
    for j in range(N):
        for k in range(N):
            C[i, j] += A[i, k] * B[k, j]
```

Implemente pelo menos **três ordenações diferentes** dos laços i, j e k, por exemplo:

`i-j-k` `i-k-j` `j-i-k`

Utilize matrizes de tamanho moderado. Compare os tempos e explique por que a simples alteração da ordem dos laços pode modificar significativamente o tempo de execução, mesmo mantendo aproximadamente o mesmo número de multiplicações, o mesmo número de somas e a mesma complexidade assintótica

$$\boxed{O(N^3)}.$$

Essa é uma das questões principais da atividade.

8. Parte VII – Questão de síntese

Questão 12

Um aluno afirma:

> "Se dois algoritmos têm complexidade $O(N^2)$, então eles terão aproximadamente o mesmo desempenho."

Você concorda? Produza uma resposta técnica de aproximadamente **10 a 15 linhas**, discutindo pelo menos constantes escondidas na notação O, organização dos dados, cache, localidade, chamadas de função, linguagem utilizada, compilador ou interpretador, vetorização e características do hardware.

9. Parte VIII – Investigação livre

Questão 13 – A máquina também faz parte do experimento

Descubra algumas características do computador utilizado na atividade:

- modelo do processador;

- número de núcleos;
- quantidade de RAM;
- cache L1;
- cache L2;
- cache L3, quando disponível.

Explique se essas características ajudam a interpretar algum dos resultados encontrados. Não é necessário instalar ferramentas específicas se essas informações estiverem disponíveis pelo sistema operacional.

10. Entrega

Entregar:

1. relatório em PDF;
2. códigos-fonte `.py` ou notebook `.ipynb`;
3. tabelas dos resultados;
4. pelo menos dois gráficos;
5. respostas às questões conceituais;
6. descrição básica do hardware utilizado.

O relatório deverá apresentar a seguinte organização:

1. Introdução
2. Metodologia
3. Hardware utilizado
4. Experimentos
5. Resultados
6. Discussão
7. Conclusões

11. Critérios de avaliação

Critério	**Peso**
Correção dos códigos e experimentos	25%
Fundamentação conceitual	25%
Análise e interpretação dos resultados	25%
Qualidade dos gráficos e tabelas	10%
Discussão crítica e conclusões	15%

Orientação importante

O objetivo da atividade **não é simplesmente descobrir qual código é mais rápido**. Para cada experimento, procure responder:

Por que esse código foi mais rápido?

Uma boa resposta deverá relacionar os resultados observados ao funcionamento do hardware e à forma como os dados são organizados e acessados. Resultados inesperados também são válidos, desde que sejam analisados criticamente.

Questão bônus – nível avançado

Escolha um dos experimentos envolvendo matrizes e utilize **Numba (`@njit`)** para compilá-lo. Repita as medições e compare Python puro, Numba e NumPy, quando aplicável.

Discuta por que o efeito da organização da memória e da cache tende a ficar mais evidente quando o custo do interpretador Python é reduzido.